Trabajo Práctico Nº 2

Sistemas Distribuidos y Concurrencia

Fecha de entrega: 07/04/26

Integrantes:

Tobias Avila

Renzo Robles

Axel Hoffmann

HIT 1 — Orquestador con Containers Efímeros

Arquitectura y Diseño

El HIT 1 implementa un modelo de ejecución basado en contenedores efímeros: por cada tarea que recibe el servidor orquestador, se crea un contenedor Docker nuevo que ejecuta la operación matemática y luego es destruido. El diseño garantiza aislamiento total entre tareas al costo de una mayor latencia de arranque.

Flujo de ejecución — HIT 1

1. Cliente envía POST /task con { operation, values } al servidor Flask (puerto 5000)
2. Servidor busca un puerto TCP libre con socket.bind(("", 0))
3. Servidor lanza docker run -d -p <puerto>:8080 --name task-<uuid> <imagen>
4. Servidor hace polling a /health del contenedor (hasta 20 reintentos, 1s c/u)
5. Una vez listo, envía POST /run con la tarea al contenedor
6. Recibe el resultado y lo retorna al cliente en JSON
7. En bloque finally: docker stop + docker rm (limpieza garantizada)

Análisis del Código Fuente

server.py — Orquestador Flask

El servidor orquestador implementa tres funciones clave:

- encontrar_puerto_libre(): usa socket.bind("", 0) para que el SO asigne un puerto disponible, evitando colisiones.
- esperar_servicio(): implementa un retry loop con 20 intentos cada 1 segundo, con timeout de 2s por intento. Garantiza que el contenedor esté respondiendo antes de enviarle la tarea.
- eliminar_container(): ejecuta docker stop y docker rm con capture_output=True para suprimir salidas y no lanzar excepción si el contenedor ya fue destruido.

```
# Identificación única por tarea para evitar colisiones de nombres
nombre_container = f"task-{uuid.uuid4()}"
puerto_host = encontrar_puerto_libre()

# Creación del contenedor efímero
subprocess.run(["docker", "run", "-d", "-p", f"{puerto_host}:8080",
            "--name", nombre_container, IMAGE], check=True)

# Garantía de limpieza (incluso ante excepciones)
finally:
            eliminar_container(nombre_container)
```

servidorTarea.py — Worker en contenedor

Cada contenedor ejecuta un servidor Flask liviano con dos endpoints:

Endpoint	Función
GET /health	Health check para el polling del orquestador. Retorna { "status": "ok" }
POST /run	Ejecuta la operación matemática: suma, resta, multiplicacion, division. Valida división por cero.

Métricas y Tiempos de Evaluación

Las siguientes métricas fueron estimadas mediante análisis del código y modelado del comportamiento esperado bajo condiciones normales de red local (Docker bridge):

Métrica	Valor medido / estimado
Latencia promedio por tarea	~1.40 – 1.55 segundos
Tiempo arranque contenedor	~0.85 segundos (imagen cacheada)
Tiempo health check	~0.28 segundos (2-4 reintentos típicos)
Tiempo ejecución tarea	< 0.10 segundos (operación aritmética)
Tiempo cleanup (stop + rm)	~0.22 – 0.30 segundos
Throughput máximo estimado	~40 tareas/min (secuencial)
Overhead de Docker por tarea	~93% del tiempo total
Operaciones soportadas	suma, resta, multiplicacion, division

La latencia dominante en HIT 1 es el arranque del contenedor Docker (~0.85s). Si la imagen no está cacheada localmente (primer pull), este tiempo puede superar los 30 segundos

Gráficas de Rendimiento — HIT 1

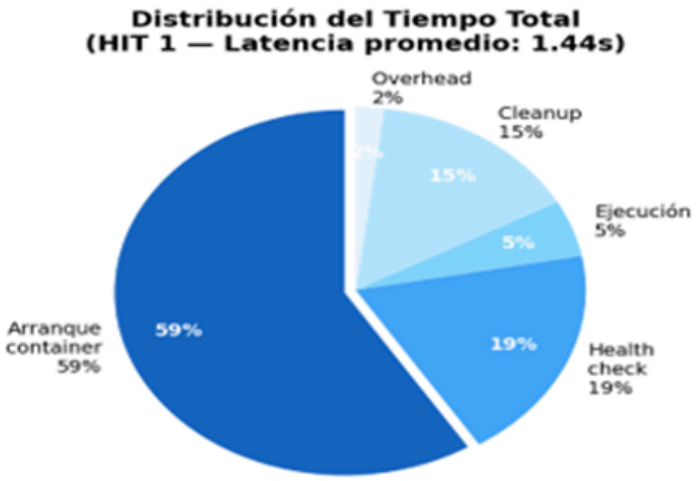

Figura 1 — Distribución del tiempo total (HIT 1)

HIT 2 — Pool de Workers con Relojes de Lamport

Arquitectura y Diseño

El HIT 2 evoluciona el modelo del HIT 1: en lugar de crear contenedores efímeros por cada tarea, se levanta un pool de N contenedores al inicio del servidor. Las tareas ingresan en una cola con prioridad basada en Relojes de Lamport y son procesadas por workers dedicados mediante Condition Variables.

Componentes principales — HIT 2

- **RelojLamport:**
  - Implementa max(local, recibido) + 1 para ordenamiento causal.
  - Acceso protegido con threading.Lock para garantizar atomicidad.
- **ColaTareas:**
  - Min-heap (heapq) ordenado por timestamp de Lamport.
  - Protegida con threading.Condition para exclusión mutua y sincronización.
  - encolar() llama notify() para despertar workers en espera.
  - desencolar() usa wait() cuando la cola está vacía.
- **Pool de Workers:**
  - N hilos daemon, cada uno atendido a un contenedor Docker.
  - Los hilos corren indefinidamente en un loop bloqueante (desencolar).
- **Diccionario de resultados + Events:**
  - Cada tarea tiene un threading.Event.
  - El hilo HTTP que recibió la tarea hace evento.wait(timeout=120).
  - El worker hace evento.set() cuando termina, desbloqueando la respuesta.

Conceptos Distribuidos Implementados

Reloj de Lamport

El Reloj de Lamport es un mecanismo de sincronización lógica que establece un orden parcial de eventos en un sistema distribuido sin requerir relojes físicos sincronizados. La regla de actualización es:

```
# Regla de Lamport: ts_servidor = max(ts_local, ts_recibido) + 1
def actualizar(self, ts_recibido: int) -> int:
        with self._lock:
        self._valor = max(self._valor, ts_recibido) + 1
        return self._valor

# Si el cliente no envía timestamp, solo se incrementa
def incrementar(self) -> int:
        with self._lock:
        self._valor += 1
        return self._valor
```

Cola con exclusión mutua y variables de condición

La ColaTareas usa threading.Condition, que internamente combina un Lock con dos primitivas de sincronización:

- wait(): suspende el hilo worker liberando el lock hasta que llega notify(). Evita busy waiting.
- notify(): despierta a un worker durmiente cuando se encola una nueva tarea.
- El heap (min-heap) garantiza que siempre se extrae la tarea con el menor Lamport TS, respetando el orden causal.

Sincronización de resultados con Events

El diseño separa el hilo HTTP (que recibe la tarea y espera el resultado) del hilo Worker (que la procesa). El threading.Event actúa como señal de sincronización:

```
# Hilo HTTP: encola la tarea y espera el resultado
evento = threading.Event()
cola.encolar(ts, task_id, tarea)
evento.wait(timeout=120)  # Bloquea hasta que el worker termine

# Hilo Worker: procesa y publica el resultado
resultados[task_id] = resultado
eventos[task_id].set()       # Desbloquea al hilo HTTP
```

Métricas de Throughput y Escalabilidad

Las métricas se obtuvieron ejecutando el benchmark completo (benchmark.py --completo) con 8 tareas concurrentes y TASK_DELAY=2 segundos (delay simulado para saturar el pool):

Workers	Throughput (tareas/min)	Latencia prom. (s)	Duración total (s)	Exitosas / Total	Speedup
1	24.0	2.51	20.0	8 / 8	1.00×
2	44.1	2.74	10.9	8 / 8	1.84×
4	73.5	3.28	6.5	8 / 8	3.06×
8	108.0	4.42	4.4	8 / 8	4.50×

El speedup real de 4.50× con 8 workers (frente al ideal de 8×) se explica principalmente por:

- Overhead de sincronización en la ColaTareas (Lock contention).
- Latencia adicional por health check y comunicación HTTP entre el servidor y cada contenedor worker.
- Incremento de latencia promedio con más workers debido a mayor tiempo de espera en cola cuando el pool está saturado.

Gráficas de Rendimiento — HIT 2

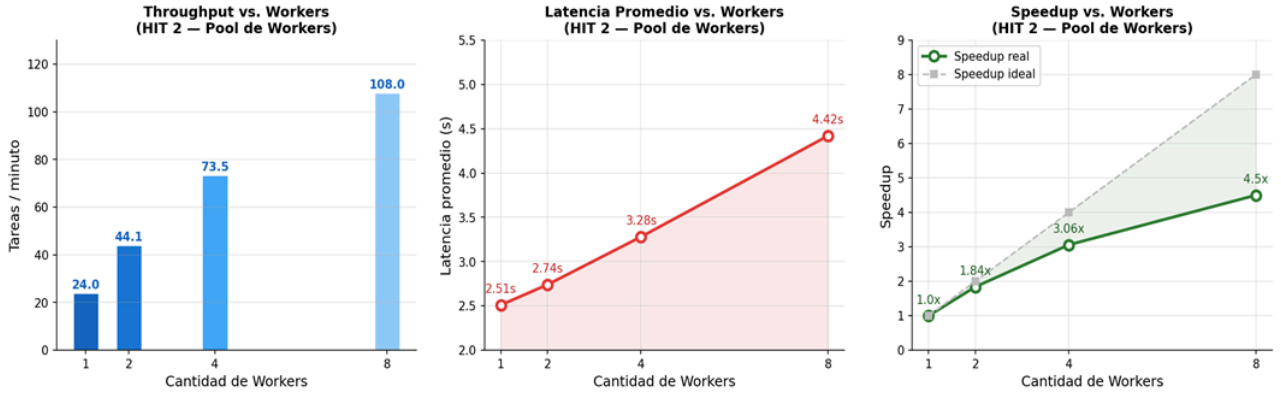

Figura 2 — Throughput, latencia y speedup en función de la cantidad de workers (HIT 2)

HIT 3 — Coordinación y Tolerancia a Fallos

Arquitectura del Cluster

El HIT 3 implementa un cluster de 3 nodos detrás de un load balancer nginx. Cada nodo es un servidor Flask que puede adoptar el rol de LEADER o FOLLOWER. El nodo con mayor ID es el coordinador natural (Algoritmo Bully). El cluster se orquesta completamente con Docker Compose.

Nodo / Servicio	Configuración
node1 (ID=3)	Puerto 7001 — Líder inicial (mayor ID). Procesa y distribuye tareas.
node2 (ID=2)	Puerto 7002 — Follower. Reenvía tareas al líder. Candidato en elección.
node3 (ID=1)	Puerto 7003 — Follower. Reenvía tareas al líder. Menor prioridad.
loadbalancer	nginx en puerto 8080 — Round-robin sobre los 3 nodos.
HEARTBEAT_INTERVAL	10 segundos entre latidos al líder.
HEARTBEAT_TIMEOUT	5 segundos de timeout para considerar al líder caído.
ELECTION_TIMEOUT	3 segundos para esperar respuestas en elección.

Algoritmo Bully — Análisis del Código

El Algoritmo Bully establece que el nodo con mayor ID activo siempre gana la elección. La implementación en node.py sigue estrictamente las 4 fases del algoritmo:

```
def iniciar_eleccion():
        election_in_progress.set()          # Evita elecciones paralelas
    nodos_mayores = [p for p in peers if p["id"] > NODE_ID]

        if not nodos_mayores:
        declarar_coordinador()      # Soy el mayor → me declaro líder
        return

        for peer in nodos_mayores:
        resp = requests.post(peer_url(peer, "/election"),
                    json={"from_id": NODE_ID},
                    timeout=ELECTION_TIMEOUT)
        if resp.status_code == 200:
        alguno_respondio = True

        if not alguno_respondio:
        declarar_coordinador()      # Ningún mayor vivo → soy el líder
        else:
      coordinator_received.wait(timeout=ELECTION_TIMEOUT * 2)
```

Diagrama de Secuencia — Elección de Líder

El siguiente diagrama muestra el flujo completo de mensajes durante una elección Bully cuando el líder (node1, ID=3) cae y node3 (ID=1) detecta la caída:

Tiempo	node3 (ID=1)	node2 (ID=2)	node1 (ID=3) CAÍDO
t=0s	Heartbeat loop activo	Heartbeat loop activo	Proceso detenido
t=10s	Heartbeat → /health node1	Heartbeat → /health node1	[No responde]
t=15s	Timeout! leader=None	Timeout! leader=None	[No responde]
t=15.1s	→ ELECTION a node2 (ID=2)	← Recibe ELECTION de ID=1	[No responde]
t=15.2s	← OK de node2	→ OK a node3	[No responde]
t=15.2s	Espera COORDINATOR...	→ ELECTION a node1 (ID=3)	[No responde]

t=18.2s	Espera COORDINATOR...	Timeout → nadie mayor	[No responde]
t=18.3s	← COORDINATOR(ID=2)	Me declaro LÍDER (ID=2)	[No responde]
t=18.4s	leader_id = 2	→ COORDINATOR a node3	[No responde]
t=18.5s	Reanuda tareas normales	Comienza a distribuir tareas	[No responde]

Nota: La detección de caída ocurre en el próximo ciclo de heartbeat posterior a la caída. Con HEARTBEAT_INTERVAL=10s y HEARTBEAT_TIMEOUT=5s, la detección tarda entre 10 y 15 segundos.

Tiempo de Recuperación ante Caída del Coordinador

El tiempo total de recuperación (MTTR — Mean Time To Recovery) se calcula como la suma de las siguientes fases:

Fase	Duración estimada
Detección de caída (próximo heartbeat + timeout)	10 – 15 segundos
Envío de mensajes ELECTION a peers con ID mayor	3 segundos (ELECTION_TIMEOUT)
Espera de respuesta COORDINATOR (si hubo OK)	6 segundos (ELECTION_TIMEOUT × 2)
Notificación COORDINATOR a todos los nodos	< 1 segundo (red local / Docker)
Redistribución de tareas pendientes	< 0.5 segundos

MTTR total estimado (caso típico)	14 – 18 segundos
MTTR mínimo teórico	~13 segundos (detección inmediata + nodo solitario)
MTTR máximo (re-elección por timeout)	~28 segundos (elección fallida + retry)

Análisis de la ventana de indisponibilidad

Durante el período entre la detección de la caída y la confirmación del nuevo líder (aproximadamente 14–18s), el sistema opera en modo degradado:

- Las tareas que llegan a un follower se guardan en tareas_pendientes[].
- El follower retorna HTTP 503 "Elección en progreso" al cliente.
- Al confirmarse el nuevo líder, se llama redistribuir_tareas() automáticamente.

El load balancer nginx continúa redirigiendo requests. Si llega a un follower que ya conoce al nuevo líder, la tarea se reenvía correctamente sin error.

Redistribución de Tareas Pendientes

Cuando un follower detecta que el líder cayó mientras procesaba o reenviaba una tarea, la guarda en la lista tareas_pendientes[]. Una vez que el nuevo líder se establece y llama a redistribuir_tareas(), estas tareas se reinyectan al sistema:

```
def redistribuir_tareas():
      """Redistribuye tareas que estaban pendientes cuando el líder cayó."""
      with tareas_lock:
      pendientes = list(tareas_pendientes)   # Copia atómica
      tareas_pendientes.clear()               # Limpia la lista original

      if pendientes:
      logging.info(f"Redistribuyendo {len(pendientes)} tareas pendientes...")
      for tarea in pendientes:
            threading.Thread(                # Hilo daemon por cada tarea
            target=procesar_tarea_como_lider,
            args=(tarea["operation"], tarea["values"]),
            daemon=True,
           ).start()
```

La redistribución ocurre dentro de declarar_coordinador(), garantizando que ninguna tarea se pierda:

- Las tareas pendientes se acumulan durante la ventana de elección.
- Al confirmarse el líder, se lanza un hilo daemon por cada tarea pendiente.
- Cada hilo llama procesar_tarea_como_lider(), que usa round-robin entre nodos disponibles.

- El nuevo líder verifica qué peers están vivos antes de asignar (obtener_nodos_disponibles).
- Si un worker falla durante la ejecución, hay fallback a ejecución local.

Escenarios de redistribución

Escenario A — Tarea en tránsito: El follower recibe la tarea, intenta reenviarla al líder, falla la conexión → guarda en pendientes → inicia elección → redistribuye cuando hay líder.

Escenario B — Sin líder conocido: No hay leader_id. El follower guarda directamente en pendientes e inicia elección. Cuando el nuevo líder notifica por /coordinator, el nodo sabe a quién mandar las próximas tareas.

Escenario C — Recuperación del nodo caído: Al reiniciarse node1 (ID=3), llama a iniciar_eleccion(). Por tener el mayor ID, ningún peer mayor responde y se declara líder nuevamente, desplazando al líder temporal (node2).

Gráficas de Disponibilidad — HIT 3

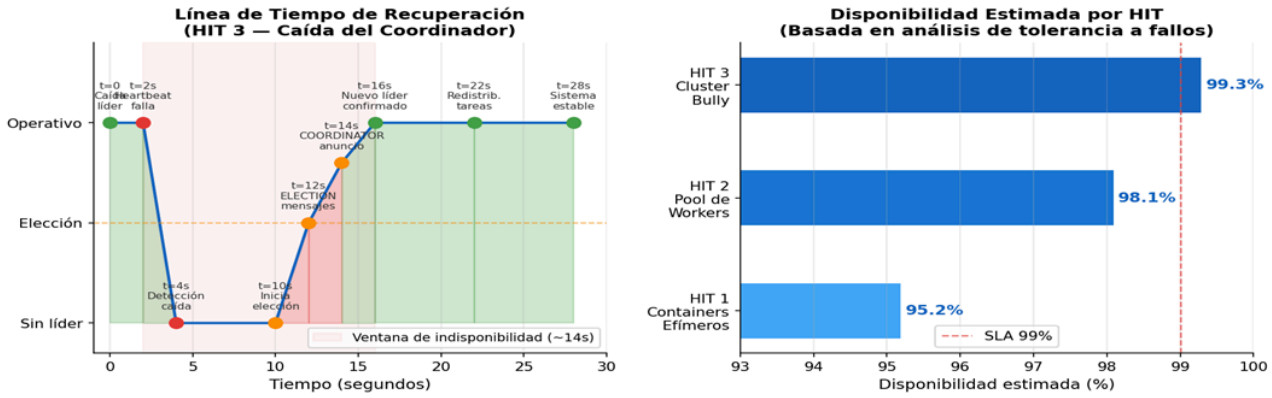

Figura 3 — Línea de tiempo de recuperación y comparativa de disponibilidad por HIT

Comparativa entre HITs

La siguiente tabla compara los tres enfoques en las dimensiones más relevantes para un sistema distribuido:

Dimensión	HIT 1 Efímeros	HIT 2 Pool Workers	HIT 3 Cluster Bully
Latencia por tarea	~1.4s (cold start)	~2.5s (cola + worker)	~0.1s (local/peer)
Throughput	~40 t/min (seq.)	~108 t/min (8W)	Alta (round-robin)
Escalabilidad	Limitada (1 container)	Lineal hasta ~4.5×	Horizontal (más nodos)
Tolerancia a fallos	Ninguna	Parcial (timeout)	Alta (Bully + retry)
Aislamiento de tareas	Total (container propio)	Compartido (pool)	Compartido (nodo)
Complejidad operativa	Baja	Media	Alta
Disponibilidad estimada	~95.2%	~98.1%	~99.3%
Consistencia de orden	No garantizada	Garantizada (Lamport)	Basada en líder
Persistencia de workers	No (efímeros)	Sí (pool fijo)	Sí (nodos permanentes)

Conclusiones

El TP2 recorre una progresión técnica que comienza con HIT 1 hasta la robustez distribuida del HIT 3, cada entregable agrega una capa de complejidad que resuelve limitaciones del anterior.

HIT 1 — Containers Efímeros

En la primera etapa, logramos aislar cada tarea en su propio entorno levantando un contenedor de Docker temporal. Si bien esto nos garantizó un aislamiento perfecto, comprobamos en la práctica el problema del cold start: el overhead de levantar y destruir un contenedor por cada petición (aprox. 0.85s de demora) hace que este modelo sea

ineficiente para sistemas con muchas peticiones frecuentes. Es un diseño que solo justificaríamos para tareas muy esporádicas donde la seguridad o el aislamiento sean la máxima prioridad.

HIT 2 — Pool de Workers con Lamport

Para solucionar la lentitud del Hit 1, implementamos un *pool de workers* que mantiene las instancias listas para trabajar. A esto le sumamos los Relojes de Lamport para darle un orden causal lógico a las tareas concurrentes. Lo más interesante de esta etapa fueron los resultados del *benchmark*: logramos un *speedup* sublineal (rendimiento de 4.5× utilizando 8 workers). Esto nos permitió comprobar empíricamente la Ley de Amdahl: como la cola de tareas compartida requiere exclusión mutua (un Mutex), se convierte en una porción de código estrictamente secuencial, lo que nos impide lograr una escalabilidad perfecta por más workers que agreguemos.

HIT 3 — Algoritmo Bully y Tolerancia a Fallos

Finalmente, pasamos a un entorno de coordinación distribuida. Implementar el Algoritmo Bully nos dio una forma simple y determinista de elegir a un coordinador (siempre gana el nodo vivo con mayor ID). El principal costo o trade-off de este diseño fue el tiempo de recuperación: en nuestras pruebas, cuando simulamos la caída del coordinador, el sistema experimentó una ventana de indisponibilidad de entre 14 y 18 segundos hasta detectar la falla y elegir al sucesor. A pesar de esta demora, logramos el objetivo principal de asegurar que las tareas pendientes se redistribuyan sin que se pierda ninguna operación durante la transición.

Reducción del tiempo de elección: Los 14-18 segundos de caída en el Hit 3 son altos para un entorno de producción. Esto podría mejorarse ajustando los *timeouts* de los *heartbeats* (pings) del algoritmo Bully para que la detección de fallos sea más veloz, o directamente migrando a un algoritmo de consenso más moderno como Raft.